

Practical-10 OUTPUT

1. Code Implementation:

Shows the C program written to implement the SSTF (Shortest Seek Time First) disk scheduling algorithm, where the program stores disk requests, initial head position, and prepares variables required to calculate seek distance and total head movement.



```
GNU nano 7.2 sstf.c *
#include <stdio.h>
#include <stdlib.h>

int main() {

    int request[] = {98,183,41,122,14,124,65,67};
    int n = 8;
    int head = 53;

    int visited[8] = {0};
    int total_movement = 0;

    printf("Order of servicing:\n");

    for(int i=0;i<n;i++){

        int min = 9999;
        int index = -1;

        for(int j=0;j<n;j++){

            if(!visited[j]){

                int distance = abs(head - request[j]);

                if(distance < min){
                    min = distance;
                    index = j;
                }
            }
        }

        if(index != -1){
            printf("%d ", request[index]);
            visited[index] = 1;
            head = request[index];
            total_movement += min;
        }
    }

    printf("\nTotal head movement: %d", total_movement);
}
```

2. Scheduling Logic:

Displays the main logic of the SSTF algorithm where the program compares the distance between the current disk head and all pending requests, selects the closest request, services it, and updates the head position accordingly.



```
GNU nano 7.2 optimal.c *
}

if(found == 0){

    int pos = -1;

    for(int j=0;j<frames;j++){
        if(frame[j] == -1){
            pos = j;
            break;
        }
    }

    if(pos == -1){

        int farthest = i;
        int index = -1;

        for(int j=0;j<frames;j++){

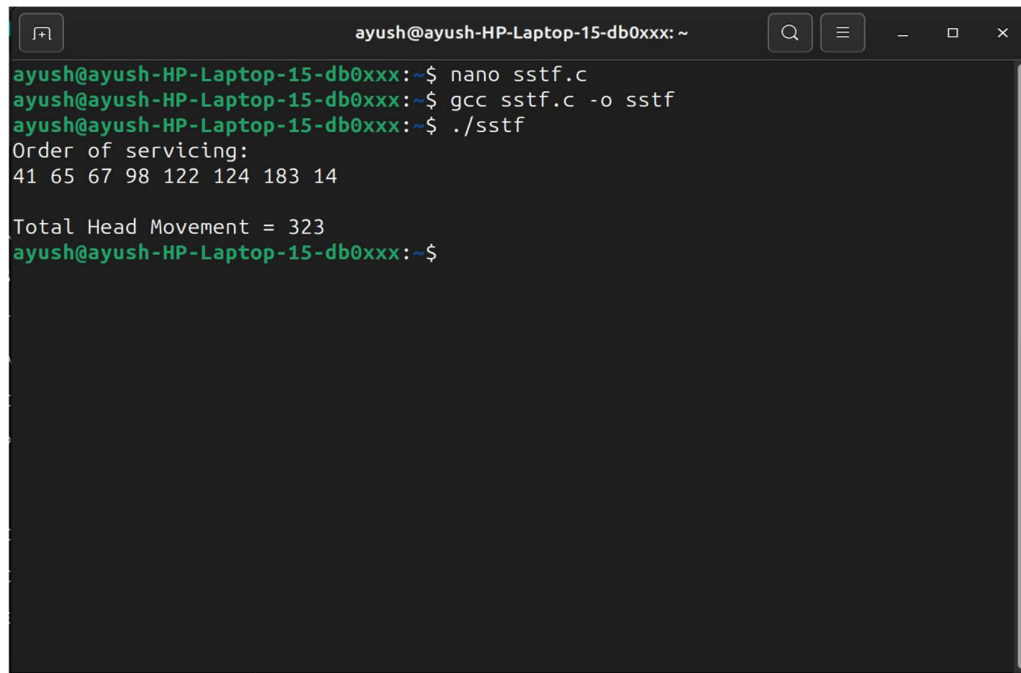
            int k;
            for(k=i+1;k<n;k++){
                if(frame[j] == pages[k])
                    break;
            }

            if(k < farthest){
                farthest = k;
                index = j;
            }
        }

        if(index != -1){
            printf("%d ", request[index]);
            visited[index] = 1;
            head = request[index];
            total_movement += min;
        }
    }
}
```

3. Program Output:

Shows the final execution result in the terminal displaying the order in which disk requests are serviced and the total head movement calculated using the SSTF disk scheduling algorithm.

A terminal window titled 'ayush@ayush-HP-Laptop-15-db0xxx: ~' with standard window controls. The terminal shows the following commands and output:

```
ayush@ayush-HP-Laptop-15-db0xxx:~$ nano sstf.c
ayush@ayush-HP-Laptop-15-db0xxx:~$ gcc sstf.c -o sstf
ayush@ayush-HP-Laptop-15-db0xxx:~$ ./sstf
Order of servicing:
41 65 67 98 122 124 183 14

Total Head Movement = 323
ayush@ayush-HP-Laptop-15-db0xxx:~$
```